

Report
Of
SMARTBRANCH 360 — SECURE BRANCH-OFFICE NETWORK

(Individual Contribution Report — Member 3: Python Tool, Documentation & Demo)

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By
Roll Number of Student: 2410030441
Name of Student: AKHIL GWARI

Batch:
2026–27

CANDIDATE'S DECLARATION

I, Akhil Gwari, do hereby declare that the internship report titled “SmartBranch 360 — Secure Branch-Office Network” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science Engineering. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own and has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

Student Name: Akhil Gwari

Roll No.: 2410030441

ACKNOWLEDGEMENT

I'd like to take this opportunity to thank everyone who supported me through the Cisco Virtual Internship 2026. I'm grateful to the faculty of the School of Computer Science and Engineering, IILM University, Greater Noida, and to the mentors at the Cisco Networking Academy for their guidance and feedback throughout this work.

I also want to thank my fellow team members on the SmartBranch 360 project — without their work on the network build and security hardening, I would not have had a verified, working network to document and validate. I'm grateful for their collaboration and the discussions that helped resolve the issues that came up along the way.

Finally, I want to thank my parents for their blessings, encouragement, and constant support throughout this internship.

Student Name: Akhil Gwari

Roll No.: 2410030441

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page No.
	Cover Page	—
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Project Description	1
	3.1 Introduction	1
	3.2 Organization Profile	2
	3.3 Problem Statement	2
	3.4 Project Objectives	3
	3.5 Scope of the Project	3
	3.6 Technologies and Tools Used	4
	3.7 System Architecture	5
	3.8 Methodology	6
	3.9 Expected Outcomes	10
	3.10 Certificates of Completion / Communication Proof	11
4.	Bibliography / References	12
	Appendix A: smartbranch_checker.py — Full Source Code	13
	Appendix B: Program Output	16

3. PROJECT DESCRIPTION

3.1 Introduction

This report has been prepared for evaluation of the internship carried out under the Cisco Virtual Internship 2026 programme, delivered through the Cisco Networking Academy at IILM University, Greater Noida. The internship combined self-paced technical training with a hands-on, team-based networking project — SmartBranch 360 — built entirely in Cisco Packet Tracer.

SmartBranch 360 is a secure branch-office network that supports wired employees, guest Wi-Fi users, an internal server, controlled Internet access through Network Address Translation (NAT), and hardened administrative management, with the whole configuration checked by a custom-built Python assurance tool. The project was done by a team of three students, with the work split across network build, security/testing, and documentation.

This report is split into three parts: first, an overview of the SmartBranch 360 project — its problem statement, objectives, and shared network topology; second, a detailed account of my individual contribution as Member 3 of the team, covering the Python assurance tool, the requirements specification, the design documentation, and the fault-injection records gathered by the team; and third, the certificates of completion earned as part of this internship.

3.2 Organization Profile

The internship was delivered through the Cisco Networking Academy, Cisco's global IT skills and career-building programme, in association with the School of Computer Science and Engineering, IILM University, Greater Noida. The Academy offers self-paced e-learning courses, hands-on labs in Cisco Packet Tracer, and industry-recognised certifications covering networking fundamentals, routing and switching, network security, and related domains.

Under the Cisco Virtual Internship 2026 programme, IILM University partnered with the Cisco Networking Academy to give students structured exposure to enterprise networking concepts. Students first completed foundational certification courses and then applied what they'd learned to a simulated branch-office networking project (SmartBranch 360), carried out in teams under faculty guidance and evaluated against a defined technical requirement.

3.3 Problem Statement

A small business needs a secure branch-office network that supports wired employees, guest Wi-Fi visitors, an internal server, controlled Internet connectivity, and restricted administrative access — while keeping guest traffic isolated from sensitive internal resources. The problem statement for the Cisco Virtual Internship 2026 project required the team to design and build this network entirely within Cisco Packet Tracer, test it thoroughly under normal operating conditions, deliberately introduce faults for troubleshooting practice, and back the whole exercise with an automated Python-based configuration

assurance tool that any team member — or an evaluator — could run to independently confirm the build was correct.

3.4 Project Objectives

- To design and implement a segmented branch-office network using VLANs for Employee, Guest, Server, and Management traffic.
- To provide inter-VLAN routing using a Router-on-a-Stick configuration and to enable automatic IP addressing via DHCP.
- To enforce security policies that isolate Guest VLAN traffic from the Server and Management VLANs using extended Access Control Lists (ACLs).
- To harden administrative access to network devices using SSH version 2, restricted to the Management VLAN only.
- To enable shared, secure Internet access for all internal VLANs using NAT overload (PAT).
- To build a machine-readable requirements specification capturing every design decision, so the build could be checked automatically rather than by manual inspection.
- To build and use a Python-based assurance tool that automatically validates the network configuration against that requirement file.
- To compile the team's design decisions, verification evidence, and fault-injection results into a single coherent design document and demonstration.

3.5 Scope of the Project

As a group project, SmartBranch 360's scope covered the full lifecycle of a small enterprise branch-office network: topology design, VLAN and IP addressing plan, Router-on-a-Stick inter-VLAN routing, DHCP services, an internal server with DNS, Guest-VLAN security enforcement, administrative access hardening, NAT-based Internet access, end-to-end functional testing, fault injection and troubleshooting, and automated assurance validation using a custom Python tool. The work was split among three team members:

Member	Area of Responsibility
Member 1	Core Network Build — VLAN creation, trunking, Router-on-a-Stick configuration, and DHCP services.
Member 2	Security, NAT & Testing — Guest VLAN ACL policy, SSH management hardening, NAT overload configuration, end-to-end testing, and fault injection.
Member 3 (Author)	Python Tool, Documentation & Demo — building the automated configuration assurance tool, authoring the requirements specification, compiling the design documentation, documenting the fault-injection cards, and recording the project demo.

As Member 3, my individual contribution — which forms the core technical content of this report — covered authoring the `requirements.yaml` specification, building the `smartbranch_checker.py` Python assurance tool used by the whole team to validate the build, compiling the shared design documentation (topology, VLAN/IP table, and security notes), documenting the five fault-injection scenarios as reusable fault cards, and recording the project's final demonstration. This work builds on the Core Network Build

completed by Member 1 and the Security, NAT & Testing carried out by Member 2, turning their combined work into a single verifiable, well-documented deliverable.

3.6 Technologies and Tools Used

- Cisco Packet Tracer — network design, simulation, and configuration platform used to build the complete SmartBranch 360 topology.
- Cisco IOS command-line interface — the source of the exported configuration text that the Python tool validates.
- VLANs and 802.1Q trunking — for logical segmentation of Employee, Guest, Server, and Management traffic.
- Router-on-a-Stick — for inter-VLAN routing on a single physical router interface.
- Python 3 — used to build the smartbranch_checker.py configuration assurance tool, using the built-in re (regular expression) module to parse exported CLI output.
- YAML (requirements.yaml) — used as the machine-readable specification of the agreed network design, against which the live build is checked.
- Word processing and documentation tools — used to compile the design document, fault cards, and this report.
- Screen recording software — used to record the 5–10 minute project demonstration.

3.7 System Architecture

The completed group topology, built in Cisco Packet Tracer (smartbranch_final_backup.pkt), consists of a Cloud/ISP simulation, an ISP router, R1 providing Router-on-a-Stick inter-VLAN routing, two access-layer switches (SW1 and SW2), an internal server, an Internet-facing server, a wireless access point, and eight or more endpoints spread across the Employee, Guest, Server, and Management VLANs. The IP addressing plan for the four VLANs is summarised below.

VLAN	Purpose	Network	Gateway
10	Employee	10.10.10.0/24	10.10.10.1
20	Guest	10.10.20.0/24	10.10.20.1
30	Server	10.10.30.0/24	10.10.30.1
99	Management	10.10.99.0/24	10.10.99.1

Within this shared topology, my individual scope of work as Member 3 covered translating this design into the requirements.yaml specification, and building the tooling and documentation needed to prove the finished network matched it.

3.8 Methodology

The project followed a build-then-secure-then-verify-then-document approach: Member 1 completed the Core Network Build (VLANs, trunking, Router-on-a-Stick, DHCP) first, Member 2 implemented the security, NAT, and testing layer, and I, as Member 3, then built the assurance tooling, compiled the documentation, and recorded the demonstration described below.

3.8.1 Requirements Specification (requirements.yaml)

The first step in my contribution was translating the team's agreed network design into a single machine-readable YAML file, so that the Python assurance tool would have one authoritative source of truth to check the live build against, rather than relying on values hard-coded into the script itself.

```
project:
  name: Smart Branch Network
  platform: Cisco Packet Tracer

vlangs:
  - id: 10
    name: EMPLOYEE
    network: 10.10.10.0/24
    gateway: 10.10.10.1

  - id: 20
    name: GUEST
    network: 10.10.20.0/24
    gateway: 10.10.20.1

  - id: 30
    name: SERVER
    network: 10.10.30.0/24
    gateway: 10.10.30.1

  - id: 99
    name: MANAGEMENT
    network: 10.10.99.0/24
    gateway: 10.10.99.1

trunk:
  interface: Fa0/1
  allowed_vlangs:
    - 10
    - 20
    - 30
    - 99

dhcp:
  pools:
    - EMPLOYEE
    - GUEST
    - SERVER
    - MANAGEMENT

security:
  guest_to_server: deny
  guest_to_management: deny
  management_ssh_to_devices: allow
  guest_ssh_to_devices: deny

nat:
  enabled: true
  overload: true

ssh:
  version: 2
  authentication: local
```

3.8.2 Python Assurance Tool (*smartbranch_checker.py*)

I designed and wrote *smartbranch_checker.py* to automatically verify the live device configuration against *requirements.yaml*, without requiring any new network design work of its own. The script reads exported “show running-config” and “show vlan/trunk” output from the router and switch, and checks each requirement using regular-expression pattern matching. It reports a PASS/FAIL line for every individual check, and a final summary count.

The script checks seven areas of the build:

- VLAN Configuration — confirms all 4 VLANs exist and are active on the switch.
- Trunk Configuration — confirms all 4 VLANs are permitted on the Fa0/1 trunk.
- Gateway Configuration — confirms each router sub-interface has the correct IP address.
- DHCP Pools — confirms each pool has the correct network and default-router values.
- Security Rules — confirms the Guest→Server and Guest→Management deny rules are present.
- NAT — confirms the PAT overload rule is configured on the correct interface.
- SSH — confirms SSH version 2 is enabled and management access is restricted to the Management VLAN.

The full source code is included as Appendix A, and a captured run of the tool's console output is included as Appendix B.

3.8.3 Design Documentation Compilation

Using the finalised configuration exported by Members 1 and 2, I compiled the shared design documentation into a single coherent report: the topology description, the VLAN/IP addressing table shown in Section 3.7, the trunking and routing notes, the DHCP configuration summary, and the security and NAT notes. This is the same design content reflected throughout this report, brought together from the separate pieces each team member built.

3.8.4 Fault Card Documentation

As part of the team's testing plan, five faults were deliberately injected into the shared configuration, one at a time, to validate that the network — and the Python assurance tool — could correctly detect and diagnose failures rather than only confirming a working build. I documented each of these as a reusable “fault card” capturing the symptom, the root cause, and the fix, summarised in the table below.

Fault	Symptom	Root Cause	Fix
Wrong default gateway	Employee PC cannot communicate outside its subnet	Incorrect gateway pushed by DHCP	Correct the DHCP default-router value
Guest VLAN removed from trunk	Guest clients lose connectivity	VLAN 20 dropped from trunk's allowed list	Re-add VLAN 20 to the trunk allowed-vlan list
Incorrect DHCP network	Guest clients get wrong/no IP addressing	Wrong network/mask in Guest DHCP pool	Correct the Guest DHCP pool network statement

DNS blocked by ACL	IP connectivity works but DNS names do not resolve	ACL denies DNS traffic to the DNS/Server VLAN	Permit DNS (UDP/TCP 53) traffic to the DNS server
NAT failure	Internal users cannot reach the Internet	NAT overload statement removed or default route missing	Restore NAT overload configuration and default route

For example, while the “Guest VLAN removed from trunk” fault was active, running the Python assurance tool immediately flagged the failure:

```

--- TRUNK CHECK ---
[PASS] VLAN 10 allowed on trunk
[FAIL] VLAN 20 allowed on trunk
[PASS] VLAN 30 allowed on trunk
[PASS] VLAN 99 allowed on trunk

RESULT: 20/21 checks passed
STATUS: SOME CHECKS FAILED

```

After VLAN 20 was re-added to the trunk's allowed-vlan list, a re-run of the checker returned 21/21 checks passed, confirming the fix — demonstrating that the assurance tool is sensitive enough to catch a single dropped VLAN on a single trunk line.

3.8.5 Verification Summary

The checker was executed against the final exported configuration files from the completed build. All 21 checks passed, confirming the build matches the requirements.yaml specification exactly.

Check	Result
VLAN Configuration — all 4 VLANs active	PASS
Trunk Configuration — all 4 VLANs on Fa0/1	PASS
Router sub-interface gateways correct	PASS
DHCP pools correctly configured (4/4)	PASS
Guest → Server traffic blocked	PASS
Guest → Management traffic blocked	PASS
NAT overload configured on outside interface	PASS
SSH version 2 enabled	PASS
SSH access restricted to Management VLAN	PASS
Overall Python assurance tool result	21 / 21 PASS

3.8.6 Demo Recording

Following Section 9 of the project brief, I recorded a 5–10 minute walkthrough of the completed SmartBranch 360 network: the topology in Packet Tracer, a live run of smartbranch_checker.py against the exported configuration, and a short recap of one fault-injection-and-fix cycle. This recording forms the project's final demonstration deliverable.

3.9 Expected Outcomes

The SmartBranch 360 project was meant to demonstrate a fully working, secure, segmented branch-office network where every design decision is backed by a machine-checkable specification rather than informal claims. That's what was achieved: the requirements.yaml file captures the agreed design, and the Python assurance tool independently confirms the live build matches it, with a documented fault-injection-and-fix cycle proving the tool can also catch real regressions.

Beyond the technical deliverable, the internship was meant to build practical, hands-on skills across the full lifecycle of a small enterprise network. Over the course of the internship, I picked up the following skills and learning outcomes:

- Translating a network design into a structured, machine-readable specification (YAML).
- Writing Python scripts that parse and validate Cisco IOS configuration output using regular expressions.
- Designing an automated assurance tool that gives clear, actionable pass/fail feedback.
- Documenting fault-injection scenarios as reusable, symptom-first troubleshooting references.
- Compiling a multi-contributor technical project into one coherent design document.
- Collaborating within a three-member team on a shared Packet Tracer topology.
- Recording a clear, structured technical demonstration of a completed project.

In conclusion, as Member 3, I was responsible for the Python Tool, Documentation & Demo on SmartBranch 360 — authoring the requirements specification, building the automated assurance tool used to validate the entire team's work, compiling the shared design documentation, documenting the fault-injection scenarios, and recording the final demonstration. This turned the network built by Member 1 and hardened by Member 2 into a verifiable, well-documented deliverable, with every design claim backed by a 21/21 automated check result. Combined with the foundational Cisco Networking Academy training completed before the project, this internship gave me practical, hands-on experience across the full lifecycle of a small enterprise network — from specification through automated verification to documentation and demonstration.

3.10 Certificates of Completion / Communication Proof

The foundational Cisco Networking Academy certificates completed as part of this internship are provided separately as the accompanying Internship Certificate document(s), consistent with the submission format for this internship.

4. BIBLIOGRAPHY / REFERENCES

- [1] Cisco Networking Academy, “Networking Essentials Course,” Cisco Systems Inc., 2026.
- [2] Cisco Networking Academy, “Getting Started with Cisco Packet Tracer,” Cisco Systems Inc., 2026.
- [3] Cisco Networking Academy, “Exploring Networking with Cisco Packet Tracer,” Cisco Systems Inc., 2026.
- [4] Cisco Systems Inc., Cisco Packet Tracer Software, Version used for SmartBranch 360 simulation, 2026.
- [5] Cisco Systems Inc., “Cisco IOS Command Reference — VLAN, Trunking, DHCP, and Routing configuration guides,” Cisco Documentation.
- [6] SmartBranch 360 — Implementation Guide (Project Team Internal Document), Cisco Virtual Internship 2026, IILM University, Greater Noida.
- [7] Python Software Foundation, “Python 3 Language Reference” and “re — Regular expression operations,” used for the SmartBranch 360 configuration assurance tool.
- [8] YAML.org, “YAML Ain't Markup Language (YAML) Specification,” used for the requirements.yaml design specification.

APPENDIX A: smartbranch_checker.py — Full Source Code

The complete source code of the Python verification tool built for this project is included below, including the main() function and script entry point.

```
import re

ROUTER_FILE = "router_output.txt"
SWITCH_FILE = "sw1_output.txt"

def read_file(filename):
    try:
        with open(filename, "r") as file:
            return file.read()
    except FileNotFoundError:
        print(f"[ERROR] {filename} not found!")
        return ""

def check_vlans(config):
    print("\n--- VLAN CHECK ---")

    expected = {
        "10": "EMPLOYEE",
        "20": "GUEST",
        "30": "SERVER",
        "99": "MANAGEMENT"
    }

    passed = 0

    for vlan_id, vlan_name in expected.items():
        pattern = rf"{vlan_id}\s+{vlan_name}\s+active"

        if re.search(pattern, config, re.IGNORECASE):
            print(f"[PASS] VLAN {vlan_id} - {vlan_name}")
            passed += 1
        else:
            print(f"[FAIL] VLAN {vlan_id} - {vlan_name}")

    return passed

def check_trunk(config):
    print("\n--- TRUNK CHECK ---")

    required_vlans = ["10", "20", "30", "99"]

    passed = 0

    for vlan in required_vlans:
        if re.search(
            rf"Fa0/1.*?{vlan}",
            config,
            re.IGNORECASE | re.DOTALL
        ):
            print(f"[PASS] VLAN {vlan} allowed on trunk")
```

```

        passed += 1
    else:
        print(f"[FAIL] VLAN {vlan} missing from trunk")

    return passed

def check_gateway(config):
    print("\n--- GATEWAY CHECK ---")

    gateways = {
        "10": "10.10.10.1",
        "20": "10.10.20.1",
        "30": "10.10.30.1",
        "99": "10.10.99.1"
    }

    passed = 0

    for vlan, gateway in gateways.items():

        pattern = (
            rf"interface GigabitEthernet0/0\.{vlan}"
            rf".*?ip address {re.escape(gateway)} "
            rf"255\.255\.255\.0"
        )

        if re.search(pattern, config, re.IGNORECASE | re.DOTALL):
            print(f"[PASS] VLAN {vlan} gateway {gateway}")
            passed += 1
        else:
            print(f"[FAIL] VLAN {vlan} gateway {gateway}")

    return passed

def check_dhcp(config):
    print("\n--- DHCP CHECK ---")

    pools = {
        "EMPLOYEE": ("10.10.10.0", "10.10.10.1"),
        "GUEST": ("10.10.20.0", "10.10.20.1"),
        "SERVER": ("10.10.30.0", "10.10.30.1"),
        "MANAGEMENT": ("10.10.99.0", "10.10.99.1")
    }

    passed = 0

    for pool, (network, gateway) in pools.items():

        pattern = (
            rf"ip dhcp pool {pool}"
            rf".*?network {re.escape(network)} 255\.255\.255\.0"
            rf".*?default-router {re.escape(gateway)}"
        )

        if re.search(pattern, config, re.IGNORECASE | re.DOTALL):
            print(f"[PASS] DHCP {pool}")
            passed += 1
        else:

```

```

        print(f"[FAIL] DHCP {pool}")

    return passed

def check_security(config):
    print("\n--- SECURITY CHECK ---")

    passed = 0

    guest_server = (
        "deny ip 10.10.20.0 0.0.0.255 "
        "10.10.30.0 0.0.0.255"
    )

    guest_management = (
        "deny ip 10.10.20.0 0.0.0.255 "
        "10.10.99.0 0.0.0.255"
    )

    if guest_server in config:
        print("[PASS] Guest -> Server blocked")
        passed += 1
    else:
        print("[FAIL] Guest -> Server rule missing")

    if guest_management in config:
        print("[PASS] Guest -> Management blocked")
        passed += 1
    else:
        print("[FAIL] Guest -> Management rule missing")

    return passed

def check_nat(config):
    print("\n--- NAT CHECK ---")

    rule = (
        "ip nat inside source list 1 "
        "interface GigabitEthernet0/1 overload"
    )

    if rule in config:
        print("[PASS] NAT overload configured")
        return 1

    print("[FAIL] NAT overload missing")
    return 0

def check_ssh(config):
    print("\n--- SSH CHECK ---")

    passed = 0

    if "ip ssh version 2" in config:
        print("[PASS] SSH version 2 enabled")
        passed += 1

```

```

else:
    print("[FAIL] SSH version 2 missing")

if "access-class SSH-MANAGEMENT in" in config:
    print("[PASS] SSH restricted to Management VLAN")
    passed += 1
else:
    print("[FAIL] SSH management restriction missing")

return passed

def main():

    print("=" * 55)
    print("    SMART BRANCH NETWORK ASSURANCE CHECKER")
    print("=" * 55)

    router = read_file(ROUTER_FILE)
    switch = read_file(SWITCH_FILE)

    if not router or not switch:
        return

    total = 0
    passed = 0

    # Switch checks
    result = check_vlans(switch)
    passed += result
    total += 4

    result = check_trunk(switch)
    passed += result
    total += 4

    # Router checks
    result = check_gateway(router)
    passed += result
    total += 4

    result = check_dhcp(router)
    passed += result
    total += 4

    result = check_security(router)
    passed += result
    total += 2

    result = check_nat(router)
    passed += result
    total += 1

    result = check_ssh(router)
    passed += result
    total += 2

    print("\n" + "=" * 55)
    print(f"RESULT: {passed}/{total} checks passed")
    print("=" * 55)

```

```
    if passed == total:
        print("STATUS: ALL CHECKS PASSED")
    else:
        print("STATUS: SOME CHECKS FAILED")

    print("=" * 55)

if __name__ == "__main__":
    main()
```

APPENDIX B: Program Output

The exact console output produced by running smartbranch_checker.py against the final router_output.txt and sw1_output.txt files is shown below.

```
=====
SMART BRANCH NETWORK ASSURANCE CHECKER
=====

--- VLAN CHECK ---
[PASS] VLAN 10 - EMPLOYEE
[PASS] VLAN 20 - GUEST
[PASS] VLAN 30 - SERVER
[PASS] VLAN 99 - MANAGEMENT

--- TRUNK CHECK ---
[PASS] VLAN 10 allowed on trunk
[PASS] VLAN 20 allowed on trunk
[PASS] VLAN 30 allowed on trunk
[PASS] VLAN 99 allowed on trunk

--- GATEWAY CHECK ---
[PASS] VLAN 10 gateway 10.10.10.1
[PASS] VLAN 20 gateway 10.10.20.1
[PASS] VLAN 30 gateway 10.10.30.1
[PASS] VLAN 99 gateway 10.10.99.1

--- DHCP CHECK ---
[PASS] DHCP EMPLOYEE
[PASS] DHCP GUEST
[PASS] DHCP SERVER
[PASS] DHCP MANAGEMENT

--- SECURITY CHECK ---
[PASS] Guest -> Server blocked
[PASS] Guest -> Management blocked

--- NAT CHECK ---
[PASS] NAT overload configured

--- SSH CHECK ---
[PASS] SSH version 2 enabled
[PASS] SSH restricted to Management VLAN

=====
RESULT: 21/21 checks passed
=====
STATUS: ALL CHECKS PASSED
=====
```